Operating Systems and Systems Programming

CPU and Memory Stress Testing Tool

A Web-Based Operating System Performance Testing and Monitoring Application

Team Members

K. Sai Samyuktha - 2520030443

V. Veda Priya - 2520030376

B. Harini - 2520030384

Under the Esteemed Guidance of

Mrs. Harika Devi

Department of Computer Science and Engineering, KLH University

Academic Year: 2026-2027

Abstract and Introduction

Abstract

The CPU and Memory Stress Testing Tool is a browser-based application that creates controlled CPU and memory workloads while monitoring the host computer. It combines a Python backend (FastAPI), multiprocessing workload engines, psutil system telemetry and a React frontend. Users can run CPU-only, memory-only or combined tests, set workload parameters and duration, watch live CPU and RAM, stop a test, and review past results.

It collects processor details, physical and logical cores, total and available memory, CPU and per-core utilization, memory and swap usage, CPU frequency (when available) and uptime. Live metrics reach the frontend through a WebSocket. Reports are saved locally as JSON and exported as JSON or CSV. Safety checks limit duration and worker count and keep a memory reserve.

OS concepts shown: process management, CPU scheduling, memory allocation, demand paging, inter-process coordination, workload cancellation, resource monitoring and performance reporting.

Background

A computer runs many processes that compete for processor time and physical memory. Stress testing deliberately raises resource demand so behaviour under load can be measured.

This tool lets users configure a test, watch live data, stop it and compare saved reports from one web interface, instead of separate terminal commands and monitoring tools.

Scope

- Targets a Windows desktop, used through a modern browser
- Backend: REST endpoints and a WebSocket stream
- Frontend: React single-page application
- Reports stored as local JSON in data/reports
- For controlled local testing and teaching, not an enterprise monitoring platform

Importance and Applications

- OS laboratories
- Introductory performance engineering exercises
- Application benchmarking experiments
- Demonstrating resource saturation and recovery
- Links measured behaviour to scheduling, multiprocessing, memory commitment and process cleanup
- Reports allow comparing runs with different settings

Target Users

- **Students:** studying Operating Systems and system programming
- **Developers:** checking app behaviour when CPU or memory is limited
- **Lab instructors:** demonstrating resource use and system monitoring
- **Technical users:** running short, controlled local performance experiments

Survey, Problem and Objectives

Existing approach	Limitation for a classroom experiment	How this project helps
Task Manager / system monitor	Good for watching, but not a full configurable test-and-report workflow	Combines workload setup, live telemetry and saved reports
Command-line stress utilities	Separate setup and manual result collection	Browser-based controls and report exports
One-time metric snapshots	Do not show how metrics change during a test	Collects time-series samples during a run
Manual spreadsheet recording	Inconsistent and time-consuming	Saves report data; JSON and CSV export

Problem Statement

Manual performance testing often needs several commands and separate monitoring tools. It is hard to set up a repeatable workload, watch CPU and memory together, stop a test safely and keep results for comparison. Uncontrolled stress can use too many resources and make a computer unresponsive.

Our solution: a central dashboard, configurable workloads, live telemetry, report history, export options and safety limits.

Objectives

1. Generate controlled CPU-only, memory-only and combined stress workloads
2. Monitor total and per-core CPU utilization
3. Display memory, swap, processor, OS and uptime information
4. Set duration, worker count, CPU intensity, memory target and monitoring interval within limits
5. Live telemetry through WebSockets, shown in dashboard charts
6. Manual cancellation and automatic completion when the duration ends
7. Keep a memory safety reserve; stop allocation when available memory is too low
8. Save test summaries and time-series metrics; export JSON/CSV
9. Demonstrate multiprocessing, CPU scheduling, virtual memory, process lifecycle and resource management

Concepts, Architecture and Flow

Process Management and Multiprocessing

A process is a running program with its own address space. The CPU engine uses Python multiprocessing to create worker processes that run CPU-bound work on many logical processors.

CPU Scheduling and Intensity

The OS scheduler shares CPU time among processes. Workers are busy for part of a cycle and sleep for the rest (duty cycle), so different intensities can be compared.

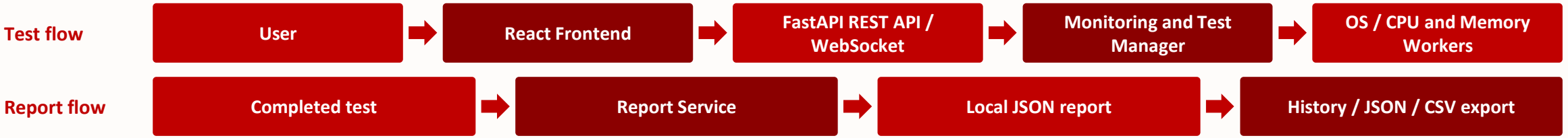
Memory Management and Demand Paging

Reserving address space is different from using real memory. The engine allocates in chunks and writes data so pages are touched, keeping a reserve for the OS.

IPC, Cancellation and Process Lifecycle

Events and shared values pass stop requests and status between manager and workers. The test manager controls the lifecycle; when a worker exits, the OS reclaims its resources.

Architecture: three layers - React frontend (presentation), FastAPI backend (service/API) and the host OS (real telemetry, runs the workers). The report service saves finished tests locally.



Flow Chart (workflow)

1. Backend starts, initializes monitoring and creates report folders

2. Frontend requests system info and metrics, connects to the metrics WebSocket

3. User picks CPU, memory or combined test and sets allowed parameters

4. Backend validates duration, worker count and memory safety budget

5. Test manager starts workload engine(s) and samples metrics regularly

6. Dashboard updates cards, charts, progress and live monitoring views

7. Test ends: duration reached, user stops it, safety limit, or error

8. Backend cleans up workers, calculates summary and saves the report

9. User reviews history, opens reports, compares runs or exports data

Modules, I/O and Technology Stack

Major Modules

- **System Monitor:** CPU, per-core, RAM, swap, frequency, uptime, host data
- **CPU Stress Engine:** CPU-bound workers, adjustable intensity
- **Memory Stress Engine:** chunked allocation, page touching, reserve check
- **Test Manager:** validates requests, manages states, workers, sampling, finish
- **Safety Configuration:** duration, worker, reserve, polling and warning limits
- **REST API:** health, system, settings, tests, history, reports
- **WebSocket Stream:** live metric snapshots at a set interval
- **Report Service:** summary stats, JSON saving, history, CSV export
- **Frontend Dashboard:** live data, setup, progress, history, comparison, settings

Input and Output Design

Inputs:

Test name and type (CPU, memory, combined)
Duration and optional warm-up
CPU worker count and intensity
Target memory in MB
Monitoring interval, safety settings
Start/stop, history, compare, export actions

Outputs:

CPU % and per-core use; RAM and swap usage
CPU frequency, uptime, system details
Test status, elapsed time, progress, workers, allocated memory
Average/peak CPU and RAM, samples, end reason
Report history; JSON/CSV downloads

Technology Stack

Python, FastAPI, Uvicorn: backend logic and workload engines; typed REST API with auto docs; ASGI server (port 8000)

psutil and multiprocessing: host telemetry; separate worker processes with shared values and events

React, Vite, Recharts: single-page UI, builds, charts

WebSockets and persistence: /ws/metrics stream with reconnect; JSON reports; avg/peak stats; CSV

Project Structure

Dev environment: backend (routes, schemas, monitoring, stress engines, reports in separate modules) and frontend (components, pages, WebSocket hook, API client). Backend files are under backend/app/.

- **main.py:** app, lifecycle, CORS, routers, WebSocket
- **config.py:** settings and safety defaults
- **api/system.py:** health, system info, metrics, settings
- **api/tests.py:** start, stop, status
- **api/reports.py:** history, details, exports, delete
- **models/schemas.py:** Pydantic schemas
- **services/monitor.py:** system info, snapshots
- **services/cpu_stress.py:** CPU worker engine
- **services/memory_stress.py:** memory worker engine
- **services/test_manager.py:** lifecycle, validation, cleanup
- **services/report_service.py:** stats, JSON, CSV
- **frontend/src/pages/:** dashboard and pages
- **frontend/src/hooks/useMetricsStream.js:** live metrics hook
- **frontend/src/services/api.js:** REST client
- **data/reports/:** persisted JSON reports

Implementation and Safety

System Monitoring

- SystemMonitor uses psutil to build snapshots: time, total and per-core CPU %, RAM, swap, CPU frequency, uptime, active workers, allocated memory, test status, ID and elapsed time
- System info: OS, Python version, CPU architecture, processor, physical/logical cores, total RAM

CPU Stress Engine

- Launches independent worker processes
- Each worker computes for part of a duty-cycle window and sleeps for the rest, following the requested intensity
- Worker count is checked against the maximum before the test starts

Memory Stress Engine

- Allocates memory in bounded chunks, not one big request
- Writes data so pages are touched and visible to the OS
- Shares allocated memory with the manager; signals when the reserve is reached
- Releases resources and ends on stop or safety abort

Test Lifecycle and Safety

IDLE > STARTING > RUNNING > STOPPING > COMPLETED / STOPPED / FAILED

- **Before start:** rejects a new request while a test is starting, running or stopping; duration at least 5 seconds and within the maximum; worker limit checked for CPU and combined tests
- **Safe memory budget:** available RAM minus the larger of the fixed reserve and a percentage of total RAM
- **Ends when:** duration is reached, user cancels, safety limit hits, or an error occurs; cleanup also runs at app shutdown
- Run stress tests only on systems you are authorized to test

Safety setting	Default	Purpose
Maximum test duration	300 seconds	Limits one test's run time
Default test duration	30 seconds	Initial duration value
Maximum CPU workers	16	Restricts worker count
Minimum available RAM reserve	1024 MB	Keeps a minimum reserve
RAM reserve percentage	15%	Reserves a share of RAM
Default monitoring interval	1.0 second	Sampling/streaming speed
CPU warning threshold	90%	Default CPU warning
RAM warning threshold	85%	Default RAM warning

Reports, Frontend Pages and API

Report Generation and Persistence

- **At test end:** average and peak CPU and RAM, peak allocated memory, actual duration, sample count, workload settings
- **Saved report:** test ID and name, type, final status, end reason, start/end time, configuration, host system info, summary statistics, metric snapshots (time series)
- Also stores CPU worker count, CPU intensity and memory target
- Stored as JSON in data/reports; IDs validated and paths checked against path traversal
- **Export:** whole report as JSON, metrics as CSV

Frontend Pages

- **Dashboard:** overview cards and charts
- **Run Test:** workload type, duration, CPU workers/intensity, memory target
- **Live Monitoring:** per-core gauges, memory/swap breakdown
- **Test History:** browse, filter, sort, inspect, export
- **Reports Comparison:** charts and comparison matrix
- **System Information:** host hardware and OS details
- **Settings:** view/update safety limits and warning thresholds
- **Active Test Banner:** progress and abort control
- **Confirmation Modal:** confirms sensitive actions

Method	Endpoint	Purpose
GET	/api/health	Backend status and version
GET	/api/system/info	Host system information
GET	/api/system/metrics	Latest metric snapshot
GET	/api/settings	Read safety settings
POST	/api/settings	Update safety settings
POST	/api/settings/reset	Restore default settings
WebSocket	/ws/metrics	Stream live metrics

API docs: http://127.0.0.1:8000/docs

Method	Endpoint	Purpose
POST	/api/tests/start	Validate and start a test
POST	/api/tests/{test_id}/stop	Stop a test by ID
POST	/api/tests/stop	Stop the active test
GET	/api/tests/status	Current state and progress
GET	/api/tests/history	List saved summaries
GET	/api/tests/{test_id}/report	Detailed report
GET	/api/tests/{test_id}/export/json	Download report as JSON
GET	/api/tests/{test_id}/export/csv	Download metrics as CSV
DELETE	/api/tests/{test_id}	Delete one report
DELETE	/api/tests/	Clear report history

Requirements and Installation

Hardware Requirements

- **Processor:** modern multi-core 64-bit CPU
- **RAM:** at least 4 GB; 8 GB+ better for memory tests
- **Storage:** space for Python/Node packages and reports
- **Display:** desktop/laptop screen for dashboard charts
- **Input:** keyboard and mouse
- **Network:** localhost; Internet to install packages
- Never set a memory test to use all installed RAM

Software Requirements

- **Windows 10/11:** target host (README)
- **Python 3.11+ (3.12 verified):** backend runtime
- **FastAPI / Uvicorn:** REST API / ASGI server
- **psutil:** CPU, memory, swap, frequency, uptime, system info
- **Pydantic v2:** validation and schemas
- **Node.js and npm:** frontend tools and packages
- **React 19 / Vite / Recharts:** UI, build, charts
- **pytest, pytest-asyncio, httpx:** backend tests
- **Vitest / React Testing Library:** frontend tests

Safe Testing Checklist

- No critical or unsaved work running
- Check free memory before a memory test
- Use a short duration for the first run
- Pick workers to match logical cores
- Keep the memory reserve at default
- Watch temperature and responsiveness
- Stop at once if the system is unstable
- Save or export the report afterwards

Installation and Execution

```
# Backend (Windows PowerShell)
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
# Frontend (second terminal)
cd frontend
npm install
npm run dev
```

Demonstration Procedure

1. Start backend and frontend (open the Vite address, usually http://127.0.0.1:5173)
2. Open the dashboard: live CPU, memory and system info cards
3. Live Monitoring: change telemetry interval if offered
4. Short CPU-only test, few workers; compare with Task Manager
5. Small memory-only test with a safe target
6. Combined test only if resources allow
7. Stop a test manually; see final status and report
8. Test History: open report, compare runs, export JSON/CSV
9. Restore default safety settings

Results, Conclusion, Future Scope

Results and Testing Strategy

- **Delivered:** 16 REST endpoints + 1 WebSocket, 7 frontend pages, 12 backend and 4 frontend tests (pytest; Vitest + React Testing Library); npm run build checks the bundle
- **Health/system APIs:** response and schema checks
- **Workload validation:** rejects bad durations, too many workers, unsafe memory
- **Start/stop, auto completion:** start, status, cancel, cleanup, final state after duration
- **Memory safety:** reserve respected; unsafe allocation rejected or stopped
- **Reports and export:** runs saved, listed; JSON/CSV endpoints
- **Frontend:** dashboard metrics, form validation, abort controls
- Actual CPU/RAM values depend on the machine used for the demo

Conclusion

The tool creates controlled CPU and memory workloads and shows system performance in a browser. The Python backend runs multiprocessing-based engines, uses psutil for telemetry, offers REST and WebSocket interfaces, and saves results as JSON with CSV export. The React interface keeps workload setup, live monitoring, history, comparison, system info and settings in one app.

It demonstrates OS concepts: processes and parallel execution, CPU scheduling and yielding, memory allocation and page commitment, resource thresholds, cancellation, process lifecycle and performance measurement. Safety limits and report history make short experiments controlled and repeatable.

Constraints and Limitations

- Results depend on host OS, permissions, hardware, background apps, power mode and thermal limits
- CPU frequency may be unavailable
- Stress can reduce responsiveness, battery life; raises temperature and power use
- Local desktop use; reports stored locally, no remote database
- No authentication or role-based access
- CORS allows all origins - restrict before wider deployment

Future Scope

- User authentication and authorization
- Report retention, backup and import
- Multiple host agents; comparison across machines
- Threshold notifications
- PDF reports and richer experiment notes
- Audit logs; deployment CORS/network limits
- Better accessibility, responsive layouts, end-to-end tests

References

- FastAPI - fastapi.tiangolo.com
- psutil - psutil.readthedocs.io
- Python multiprocessing - docs.python.org/3/library/multiprocessing.html
- Pydantic - docs.pydantic.dev
- React - react.dev; Vite - vite.dev
- Recharts - recharts.org
- pytest - docs.pytest.org; Vitest - vitest.dev
- Project source archive (backend, frontend, README, tests, sample reports)

Thank you | We thank our guide Mrs. Harika Devi, the department, and the open-source tools used (Python, FastAPI, psutil, React, Vite, Recharts)

THANK YOU